

# Building Service Layers Above Research Cyberinfrastructure for Governed Storage and Reusable Analysis

Luis Fernando Palacios Flores<sup>1,2</sup>, Federica Bazzocchi<sup>1,\*</sup>, and Tommaso Rodani<sup>1,\*</sup>

<sup>1</sup>AREA Science Park, Research and Technology Institute,  
Padriciano 99, 34149 Trieste, Italy

luis.palacios@areasciencepark.it; federica.bazzocchi@areasciencepark.it; tommaso.rodani@areasciencepark.it

<sup>\*</sup>Corresponding authors.

<sup>2</sup>SISSA – International School for Advanced Studies,  
Via Bonomea 265, 34136 Trieste, Italy

**Abstract**—Research facilities produce laboratory data meant to remain FAIR — Findable, Accessible, Interoperable, and Reusable. After acquisition, that intention often fractures: files scatter across personal drives, access is granted informally, and downstream analysis depends on ad hoc file handoffs. We present two service layers above the cyberinfrastructure of Nano Foundries and Fine Analysis — Digital Infrastructure (NFFA-DI) that add those missing rules for FAIR compliance. *Buckets Explorer* is a user friendly web application targeted at FAIR data governance, enforcing tenant scope, research proposal lifecycle, role-based sharing, and file provenance at the storage layer above Ceph object storage. The second layer is a reusable asynchronous analysis service and here we instantiate it on a Scanning Electron Microscopy (SEM) image classifier. The *SEM Classifier API* is a JSON Web Token (JWT)-authenticated application programming interface (API) for image submission, job polling, and result retrieval, while keeping the contract, queue lifecycle, and request record stable as the machine learning (ML) model and inference logic evolve. Both layers were implemented on the Open Research Facility for Epigenomics and Other (ORFEO), AREA Science Park’s research data center in Trieste, where *Buckets Explorer* is live and the SEM service is deployed and validated. From these implementations, we identify a shared service-layer pattern for research facilities: a public contract, an authentication boundary, a control plane, an infrastructure data path, and a persistent service record. We release both codebases as open-source software for inspection, reuse, and adaptation.

**Index Terms**—FAIR data, research cyberinfrastructure, service layers, governed object storage, asynchronous inference, research data management

## I. INTRODUCTION

The FAIR principles [1] describe how research data should remain usable after production, but the difficult part often begins once laboratory acquisition ends. A facility may already run object storage, compute platforms, federated login, and proposal-management systems. These systems can hold files, execute services, authenticate users, and record accepted projects, but they do not by themselves translate project membership into storage roles or turn analysis code into a stable way to submit work, check progress, and retrieve results. When that layer is missing, scientists fall back to personal

cloud drives or ad hoc file handoffs. These workarounds may be convenient, but they move data and access decisions outside the facility’s governed infrastructure. FAIR then remains a recommendation rather than a property enforced by the facility infrastructure.

This work is set in NFFA-DI [2], an Italian initiative that connects materials-science operational units through a shared digital ecosystem. Within NFFA-DI, the Overarching FAIR Ecosystem for Data (OFED) [3] organizes the path from instruments to post-acquisition services. The deployment used in this paper runs on ORFEO [4], whose services include Kubernetes container orchestration, Ceph object storage, and Authentik, the shared OpenID Connect (OIDC) identity provider used by the two implementations. Authentik supplies institutional identity claims; the service layer then applies local rules for storage roles and analysis access.

This context makes two gaps concrete. For storage, scientists need governed access tied to proposal membership, tenant scope, sharing roles, and upload provenance. For analysis, portals, notebooks, and downstream pipelines need a stable request lifecycle that remains usable even when the model implementation changes. ORFEO provides the infrastructure capacity; the service layer adds the policy and execution rules that make that capacity usable in facility workflows.

We implement this service layer twice above shared ORFEO infrastructure, at two points in the OFED data lifecycle where FAIR-enabled laboratory pipelines need post-acquisition services: governed storage for prepared data and reusable analysis for downstream consumers. Figure 1 positions both instantiations in that lifecycle, between the pipelines that prepare data and the clients that consume governed storage or analysis services.

### Contributions.

- 1) We introduce *Buckets Explorer*, a user-facing governed object-storage layer above Ceph that gives authorized users a web workspace while enforcing tenant scope, proposal lifecycle, role-based access, and file provenance.

- 2) We present the *SEM Classifier API*, a reusable asynchronous inference backend above Kubernetes that frontends, notebooks, and pipelines can call through a stable JWT-authenticated submit-poll-retrieve contract, with per-user request records and an interchangeable domain-specific payload.
- 3) We extract a shared service-layer pattern from the two instantiations; Section V defines the common responsibility split among client-visible public contract, authentication boundary, control plane, infrastructure data path, and persistent service record.

The Buckets Explorer and analysis-service codebases are available at [github.com/RitAreaSciencePark/s3bucket-manager-app](https://github.com/RitAreaSciencePark/s3bucket-manager-app) and [github.com/RitAreaSciencePark/sem-image-classifier-api](https://github.com/RitAreaSciencePark/sem-image-classifier-api), respectively. The NFFA-DI storage service is live at [buckets-explorer.areasciencepark.it](https://buckets-explorer.areasciencepark.it).

## II. RELATED WORK

Existing systems address several neighboring parts of the research-data lifecycle, including metadata discovery, cross-site data management, workflow execution, reproducible packaging, and delegated authorization. This paper focuses on a narrower layer: the facility-facing services that turn local infrastructure into governed object storage and reusable asynchronous analysis execution.

**Metadata and discovery platforms.** Deriva-ML [5] approaches machine-learning investigations as FAIR, reproducible pipelines by linking datasets, workflows, and model assets. NOMAD [6] provides a materials-science platform for managing, sharing, publishing, and discovering scientific data. These systems are most useful once data can already be treated as a managed dataset with sufficient structure and context. The storage layer in this paper addresses an earlier operational point: the workspace where research objects are first named, written, shared, and governed under facility policy.

**Cross-site data management.** Other systems move, federate, or govern data across distributed environments. Globus [7] supports secure transfer, synchronization, and sharing between research endpoints; ESGF [8] provides federated access to distributed climate and geospatial data; Rucio [9] manages scientific data across heterogeneous distributed storage; iRODS [10] provides policy-based data management; and OneData [11] provides uniform access to data in organizationally distributed environments. These systems are natural downstream targets once data has been prepared for registration, federation, transfer, or policy-driven management. The storage layer presented here remains local to one facility deployment and applies proposal membership, tenant naming, and bucket-level access before data leaves the facility workspace.

**Workflow and gateway systems.** Workflow and gateway systems orchestrate scientific computation across local, cloud, or distributed resources. Pegasus [12] maps scientific workflows onto distributed computing infrastructures; JAWS [13] provides a centralized workflow execution framework for FAIR scientific workflows; FaaS [14] supports scientific workflows through function-as-a-service infrastructure; and Airavata [15] provides

science-gateway abstractions for managing applications and workflows on backend computing resources. These systems decide how larger computations are composed, scheduled, and executed. The analysis layer in this paper is smaller in scope: it exposes one authenticated asynchronous inference service that a workflow engine, portal, notebook, or downstream pipeline could call as a step. The present deployment does not claim integration with those workflow systems.

**Reproducible packaging and authorization.** Backpacks for Notebooks [16] addresses portability by packaging notebooks with the execution environments needed to run them in distributed settings. Here, the reusable unit is different: a service template whose gateway, queue lifecycle, authentication boundary, request record, and deployment structure can remain stable while the domain-specific inference payload changes. SciTokens [17] addresses delegated authorization for distributed scientific computing through capability-bearing tokens. The services presented here use OIDC claims at the boundary and derive authorization from local tenant, proposal, role, or endpoint policy inside the service layer.

## III. BUCKETS EXPLORER: A GOVERNED-STORAGE SERVICE LAYER

### A. Storage Governance Problem

A research facility needs a workspace that gives users a simple path to governed storage without moving data outside the infrastructure responsible for FAIR policy. Two data lifecycles meet in that workspace. Data produced under an accepted facility *research proposal* carries the proposal's membership and access rules: instrument scientists at the host site need read-write access, visiting proposal members get read-only access, and only facility members may delete or re-share the data. Data produced inside a laboratory for in-house work has no such external mandate: whoever creates the workspace owns it. A researcher should not have to translate those lifecycles into storage administration. Ceph separates tenants and enforces storage permissions, but its generic controls do not by themselves encode research proposal membership, ownership of in-house workspaces, institutional naming, or file-level provenance like what is needed for FAIR compliance in research facilities such as those in NFFA-DI. The service layer maps those scientific rules onto storage operations.

### B. Service Model and User Workflow

Buckets Explorer is a Kubernetes-deployed web application that gives each authorized user a governed workspace for the storage namespaces they can use. The service groups access under a *tenant*, the administrative scope for a facility unit or cross-institution collaboration. Users who belong to multiple tenants can switch tenant context before browsing storage. Within a tenant, a *bucket* is a named object-storage namespace governed by one access policy. A *proposal bucket* is synchronized from an upstream proposal-management source; that source supplies membership and roles for accepted project data. A *local bucket* is a user-created workspace for in-house work, created interactively by a write-capable tenant member

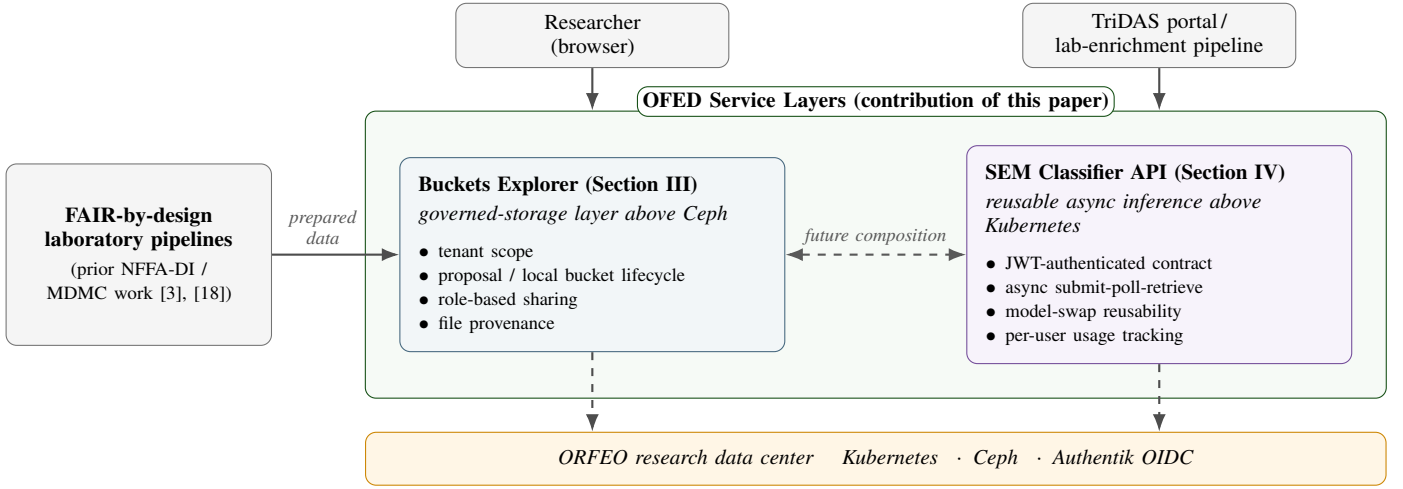


Fig. 1. OFED service-layer catalogue contributed by this paper. Two independently deployable service layers run above shared ORFEO infrastructure: *Buckets Explorer* (Section III) governs object storage above Ceph, and the *SEM Classifier API* (Section IV) exposes reusable inference above Kubernetes. Prior FAIR-by-design laboratory pipelines feed prepared data into governed storage; NFFA-DI users reach *Buckets Explorer* through a browser; clients such as TriDAS or laboratory enrichment pipelines can call the analysis API. The layers share identity and infrastructure. Each layer retains its own contract. The dashed link denotes possible future composition, not current behaviour.

with the creator as owner. Users browse proposal and local buckets, upload and download objects, and share local buckets through one interface. Proposal buckets keep the lifecycle imposed by the governance source; local buckets follow the owner’s policy. In Figure 2, authorized users see project-oriented names and only the actions allowed by their roles. The interface keeps storage identities and policy state behind that project view. The concrete source for proposal data, the institutional-identifier mapping, and the Authentik groups that open each tenant are configurable; the production setting in this paper is the NFFA-DI deployment on ORFEO.

*Buckets Explorer* exposes two surfaces over the same application state. The researcher workspace includes tenant selection, bucket dashboards, proposal and local bucket browsing, file upload and download, local-bucket sharing, identity-profile inspection, tenant documentation, and inline file viewing. The operator administration interface includes tenant activation, user and bucket inspection, institutional-unit mappings, storage-control readiness, file-format summaries, and tenant documentation management. Together, those pages keep researcher actions, operator activation, and application state aligned.

### C. Storage-Control Architecture

Figure 3 separates the application, storage-control, and object data paths. The researcher interacts with *Buckets Explorer* through a browser; every service component runs inside the ORFEO Kubernetes cluster. The frontend speaks to the Django web-application backend over a REST API authenticated using OIDC JWTs [19]–[21]. The backend authenticates each call against Authentik, decides whether the operation is permitted, and records application state in a PostgreSQL relational database instance. Two distinct paths leave the backend toward storage. The control path goes

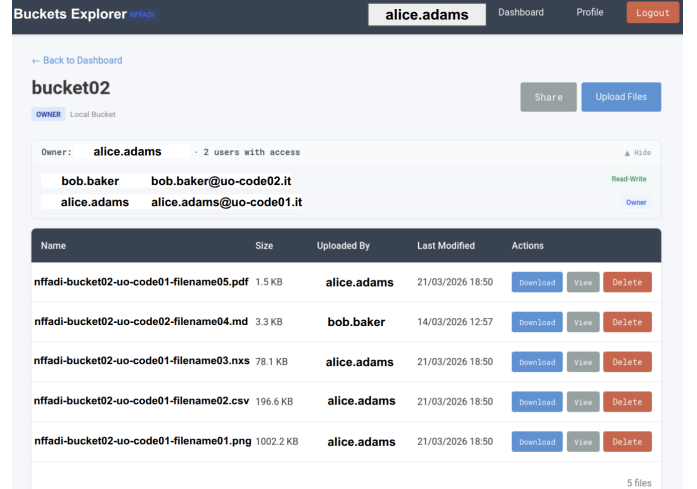


Fig. 2. *Buckets Explorer* user workspace. Authorized users see the buckets available to them and the actions allowed by their role; tenant, naming, and provenance rules are enforced by the service rather than by manual storage handling.

through RGWSquared, ORFEO’s storage-control microservice for tenant, user, bucket, and permission management. The object path bypasses RGWSquared. Upload, download, and listing use Ceph RADOS Gateway (Ceph RGW) over the Amazon Simple Storage Service (S3)-compatible HTTP protocol, but Django still authorizes the workflow and records the metadata needed for governance and provenance.

**Authentik** [19], [22] anchors the identity side of this architecture. OIDC is built on OAuth 2.0 (OAuth2), and Authentik can delegate authentication to institutional identity services that participate in a tenant or collaboration. ORFEO already uses Authentik for single sign-on (SSO), so *Buckets Explorer* does not create a second account system for facility

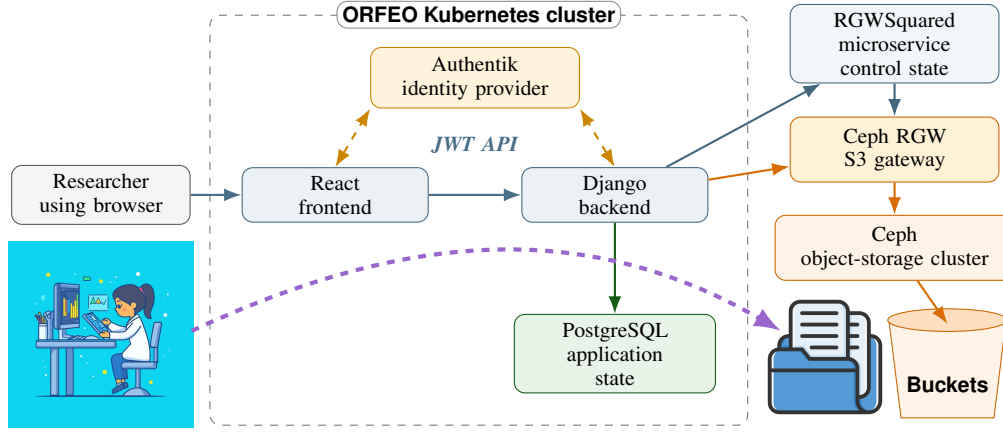


Fig. 3. Responsibility split in the Buckets Explorer architecture. The researcher uses a browser outside the ORFEO Kubernetes cluster; the React frontend, Django web-application backend, Authentik identity provider, and PostgreSQL relational database form the application layer. Django authorizes tenant and bucket operations, uses RGWSquared as the storage-control microservice, and reaches Ceph RGW for S3 operations on buckets and objects stored in the Ceph cluster. The dashed amethyst arrow marks object traffic bypassing RGWSquared; the same workflow still passes through application authorization and records governance and provenance state.

users.

The application layer keeps governance state separate from object storage. **Django**, an open-source Python web framework, supplies the backend, object-relational mapping, and operator administration surface. **PostgreSQL** stores tenants, tenant memberships, buckets, per-bucket roles, file upload records, and the policy flags that the application must evaluate before issuing storage calls. For example, `bucket_type` distinguishes proposal buckets from local buckets, and `is_deletable` prevents proposal-governed buckets from being treated like local workspaces. The browser UI is built with **React**, an open-source JavaScript library. It presents only actions authorized by Django, so a researcher or operator sees the same policy state that the backend will enforce.

The storage layer keeps object traffic on infrastructure that ORFEO already operates. **Ceph** [23], [24] is the production object store, and Ceph RGW exposes the S3-compatible data plane. **RGWSquared** centralizes Ceph governance for ORFEO services. Instead of exposing Ceph administration directly to each application, services issue governed requests such as “give user  $U$  role  $R$  on bucket  $B$ ”, and RGWSquared translates them into Ceph subusers, buckets, permissions, and credentials. For NFFA-DI, it also enforces proposal-derived bucket visibility by synchronizing Ceph permissions with upstream proposal state. RGWSquared manages bucket lifecycle, permissions, structure readiness, and transient S3 credential issuance; Django keeps the UI-facing records for tenant membership, bucket sharing, file uploads, and institutional-identifier mappings. This division is an ORFEO implementation choice, not a portability requirement: another facility could place the same governance operations in a different adapter or inside the application itself. The portable design principle is narrower: keep lifecycle and permission decisions separate from the high-volume object path, so upload and download traffic do not require the governance component to carry object payloads.

#### D. Governed Storage Capabilities

1) *Tenant-Gated Workspace and Operator Readiness*: At login time, authentication is necessary but not sufficient to open a storage workspace. After Authentik authenticates an account, Buckets Explorer checks tenant readiness before exposing storage. A tenant is ready only when its Django record is active, the user belongs to the Authentik group mapped to that tenant, the storage-control structure is initialized on Ceph through RGWSquared, and institutional-identifier coverage is available when institutional codes are required. An account can enter one ready tenant and still be blocked from another tenant that has not passed the readiness checks. Operators manage that readiness from the administration interface: they create or activate the tenant, add the group mapping, refresh users and buckets from the storage-control service, inspect institutional-identifier coverage, and check whether the service is ready for users. Tenant activation is an explicit facility operation rather than a side effect of login.

2) *Proposal/Local Lifecycle and Sharing*: Buckets Explorer keeps official proposal storage and user-created workspaces under different lifecycle rules. Proposal buckets inherit membership and roles from upstream project state synchronized through RGWSquared, and the application refuses local delete and re-share requests on those buckets. Local buckets are created by write-capable tenant members, who keep owner responsibility for the workspace data and can share access with other tenant members through local policy-mediated grants. Three roles cover the allowed operations: an owner controls the lifecycle of a local workspace, a read-write collaborator can upload and download, and a read-only collaborator can only view and download. Table I lists the assignments. Owner powers apply to local buckets only. These restrictions reflect the data-policy obligations of a research facility, where over-broad permissions on a shared workspace can expose data beyond the authorized collaboration. Every role decision is evaluated

TABLE I  
PERMISSION MATRIX: ROLE-OPERATION ASSIGNMENTS IN BUCKETS  
EXPLORER

| Role  | Create local | Delete local | Upload | Download | Share local |
|-------|--------------|--------------|--------|----------|-------------|
| Owner | ✓            | ✓            | ✓      | ✓        | ✓           |
| RW    | —            | —            | ✓      | ✓        | —           |
| RO    | —            | —            | —      | ✓        | —           |

in Django before any storage call is issued, and RGWSquared receives only the storage permissions that match the authorized role.

3) *Policy-Carrying Names and Upload Records*: In this layer, names carry provenance into ordinary storage listings, and access rules fix authorization before an object is written. Bucket names and object keys are policy, not free text. Proposal buckets keep the upstream identifier issued by the facility governance source, and their display names are synchronized from that source rather than generated from a local owner. Local buckets are assembled from the username, the institutional *operational-unit (UO) code* (a short, stable identifier of the user’s home institute), and the user-provided project label. For example, a local project label can produce `alice-adams-cnr-iom-ts-graphene-sample`. Users type only the project label; the service generates a lowercase, hyphen-separated, S3-safe bucket name. That generated name avoids collisions when two users choose the same visible label inside a tenant. UO codes are optional per-tenant mappings loaded from a YAML fixture. Single-institution tenants leave the fixture empty and fall back to `username-project`; tenants that federate multiple institutions populate the fixture once. In the case of NFFA-DI, the fixture covers eleven operational units [25].

The same naming discipline extends to every uploaded object. Before a file is written to Ceph, Buckets Explorer composes an S3-safe lowercase object name that embeds the same institutional signal as the bucket name. For NFFA-DI proposal buckets the object-name pattern is `tenant-bucket-uploader-uo-filename`; for NFFA-DI local buckets the order is inverted to `tenant-uploader-uo-bucket-filename` [25]; for tenants without UO codes the pattern is `tenant-bucket-filename`. The UO segment is taken from the *uploader*, not from the bucket owner, so a shared read-write upload still carries the institutional signal of whoever actually wrote the object.

Every upload creates an application-level record containing the stored object name, uploader, file size, and timestamp. Those records enforce per-file deletion policy: a local-bucket owner can remove any file, and a read-write collaborator can remove only files they uploaded. Storage activity remains reviewable without modifying Ceph itself. The record is also the bridge between the friendly bucket view and the object name stored in Ceph. The researcher sees the friendly bucket view rather than generated object names. Operators can trace an object back to the upload that created it. Operators can

also configure tenant-level file-name rules, inspect files that deviate from those rules, and review file-format summaries by count and storage use. Those administration views make policy drift and storage composition visible from the application layer instead of requiring manual inspection of Ceph objects.

4) *Authorized Data Path and Scientific File Access*: The two storage paths in Figure 3 keep governance at lifecycle and permission boundaries. RGWSquared mediates create, delete, grant, revoke, and synchronization events; Ceph RGW carries object payloads over S3 [26]. Before an object operation is offered to the user, Django has already checked the tenant, bucket, role, and file-level rule that applies to the request. Sending file bytes through RGWSquared would not add another policy decision, so object payloads use the S3-compatible path; lifecycle and permission changes stay on the RGWSquared control path. Once that authorization step has run, the workflow gives scientists a working view of stored data: users can browse buckets, upload and download objects, and preview supported file formats in the browser. For OFED scientific data, that includes NeXus/HDF5 files through the dedicated viewer when the file type is supported. Downloads remain available when inline preview is not supported. Table I maps each role to its permitted storage actions.

#### IV. SEM CLASSIFIER API: A REUSABLE ASYNCHRONOUS ANALYSIS SERVICE LAYER

##### A. Reusable Analysis Problem

Research facilities increasingly need to expose analysis methods as services, not only as scripts, notebooks, or model files. A trained ML model may solve a domain task, but it does not by itself provide the operational interface needed by facility users and downstream systems. To become a usable service, each analysis must also provide authenticated access, input validation, asynchronous execution, job-state tracking, result retrieval, and per-user accounting.

The recurring work is not the scientific inference itself, but the service machinery around it: gateway configuration, queue handling, authentication checks, deployment files, and usage records. If that machinery is rebuilt for every model, each payload becomes a separate service implementation. Instead, the service layer should make the operational path repeatable, so a new inference service can be produced by adapting the shared logic and deployment configuration while replacing only the domain-specific parts: model loading, input handling, inference logic, and runtime resources.

This template can support different clients, including facility portals, notebooks, workflow engines, and downstream laboratory pipelines. Trieste Advanced Data Services (TriDAS) [27], a microscopy-oriented portal and service catalogue, is one example client. The SEM Classifier API is the instantiation of this reusable asynchronous analysis-service template presented in this paper.

##### B. Service Contract and Execution Flow

The client-facing contract exposes a simple asynchronous workflow: submit an SEM image by URL or upload, receive

a job identifier, poll the job state, and retrieve the result. This submit-poll-retrieve pattern lets inference run without blocking the caller. Access is authenticated with JWTs issued by Authentik, and each authenticated request creates a per-user request record.

In the deployed instantiation, the domain payload is `t0m-R/vit-sem-classification`, a Vision Transformer (ViT) model fine-tuned on ten material classes from the NFFA-DI SEM dataset [28]–[30]. ViT-B/32 is the base-size ViT variant that processes images as 32-pixel patches; fine-tuning adapts the pretrained image model to the NFFA-DI SEM classes. This payload is SEM-specific, while the surrounding service logic is reusable.

Figure 4 maps the request flow onto the deployment. The client remains outside the ORFEO Kubernetes cluster; the gateway, queue, stores, and workers run inside it. After the JWT issuer authenticates the client and issues a signed token, the client submits an SEM image to KrakenD. KrakenD checks the route policy, forwards authorized requests to BentoML, and returns a job identifier without waiting for inference to finish. BentoML executes the job asynchronously. Redis stores the pending queue, job state, and completed result for later status and result calls.

Request accounting is kept separate from inference state. A Go plugin in the gateway writes one row to the PostgreSQL `api_usage` table for each authenticated submit, status, and result request. Redis keeps transient inference state, with job entries expiring after a configurable one-hour time-to-live (TTL). PostgreSQL keeps the durable per-user activity record, but does not store model outputs.

### C. Reusable Template Architecture

The reusable template is the boundary that separates service machinery from domain-specific inference. **BentoML** [31] packages the Python payload as a versioned HTTP service. Around it, the shared template provides gateway configuration, JWT checks, asynchronous queue handling, result retrieval, request accounting, and deployment files.

**Redis** [32] stores transient job state and cached results, while PostgreSQL persists only request-level records in the `api_usage` table. A new analysis service can therefore reuse the same operational path and adapt only the payload-specific parts: model loading, input handling, inference logic, and runtime configuration. Endpoint routing, authentication, queue lifecycle, and request records remain stable unless the public contract is intentionally changed.

### D. Operational Capabilities

1) *Async Submit-Poll-Retrieve Contract*: The public contract exposes three endpoints, summarized in Table II. `/api/v1/inference` accepts the submit request and returns a job identifier. `/api/v1/jobs/status` reports the current job state. `/api/v1/jobs/results` returns the completed result. All three endpoints use `POST` because the BentoML backend receives JSON request bodies.

TABLE II  
PUBLIC API CONTRACT EXPOSED BY KRAKEND, STABLE ACROSS MODEL VARIANTS. ALL ENDPOINTS USE `POST` BECAUSE BENTOML RECEIVES JSON REQUEST BODIES.

| Endpoint                          | Response schema                |
|-----------------------------------|--------------------------------|
| <code>/api/v1/inference</code>    | job_id, status message         |
| <code>/api/v1/jobs/status</code>  | job state and timestamps       |
| <code>/api/v1/jobs/results</code> | result, error, completion time |

2) *Model-Swap Reusability*: The service separates reusable execution behavior from domain-specific inference through the three-level class hierarchy in Figure 5. `BaseAsyncModelService` implements the common asynchronous lifecycle: Redis job management, status and result lookup, health checks, the worker loop, and hooks for serializing client inputs into Redis-safe records and reconstructing them before inference. `ImageAsyncModelService` adds image-specific adapters for URL loading, uploaded-file decoding, and image-size validation. `SEMInferenceRedisService` supplies the deployed ViT-B/32 SEM payload.

This split defines what changes when the service is adapted. Another image model can keep `ImageAsyncModelService` and replace only the model-loading and inference methods. A non-image model can keep `BaseAsyncModelService` and provide a different input adapter. In both cases, the KrakenD endpoints, Redis lifecycle, authentication boundary, and `api_usage` schema remain unchanged unless the public contract is intentionally revised. Rodani’s SEM-scale and STM-artifact models fit the same image-payload path [33]–[35]; they are cited as possible replacement payloads, not as additional deployed services.

3) *Per-User Usage Tracking*: The PostgreSQL `api_usage` table records timestamp, username, service name, endpoint type, status code, and URL path for every authenticated request. Operators use it to review service activity, plan capacity, and detect abuse. A frontend such as TriDAS can run on top of this backend and receive a per-user activity record without re-implementing request logging. The table is intentionally limited to request metadata; the classification result remains in Redis until the job’s TTL elapses. This keeps transient inference state separate from persistent usage records.

4) *Device Deployment*: The current validation deployment serves the ViT-B/32 model on a central processing unit (CPU). We characterized this deployment on the ORFEO validation cluster using a fixed public SEM JPEG URL, one BentoML replica, and three independent trials per load setting. At concurrency 1, the 90-job run completed with 100% success, with median end-to-end latency of 2.8 s and median server-side processing time of 0.6 s. At concurrency 10, the 150-job run completed with 100% success, reaching 36.9 jobs/min. Median end-to-end latency increased to 15.1 s, with p95 at 21.9 s, while median processing time remained

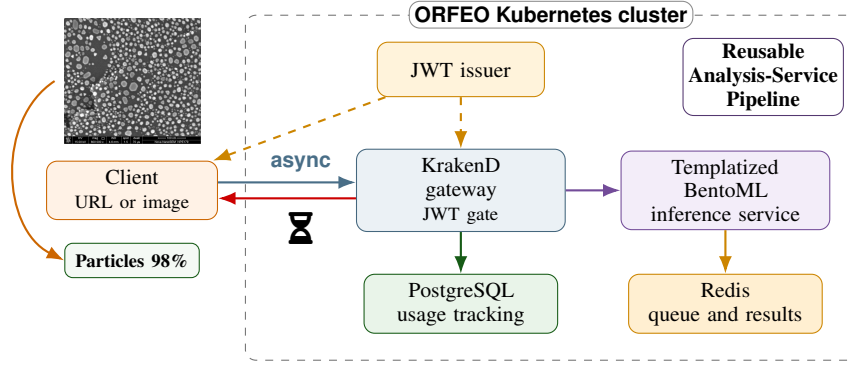


Fig. 4. Logical architecture of the reusable analysis-service pipeline, instantiated by the SEM Classifier API. The client submits either a URL pointing to a SEM image or an image file directly to the gateway. The client remains outside the ORFEO Kubernetes cluster. The JWT issuer handles authentication, and KrakenD, the public API gateway, checks requests before delegating work to BentoML model serving, Redis queue storage, and PostgreSQL request records. The blue forward arrow is the submit request; the red return arrow carries the job identifier used by later status and result calls.

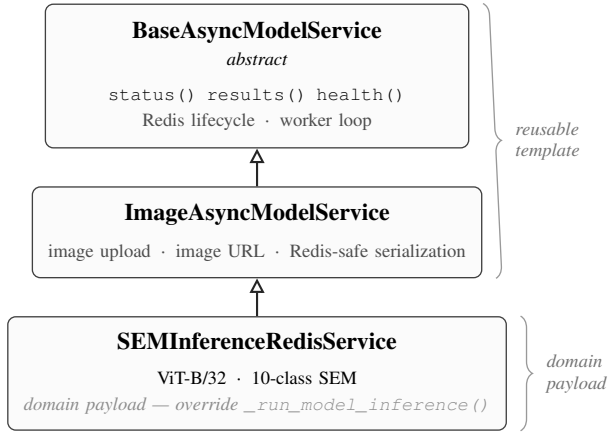


Fig. 5. Three-level class hierarchy for the reusable analysis service. Open-triangle arrows denote inheritance. The deployed SEM model occupies the domain-payload layer; queue lifecycle, public endpoints, and request tracking remain outside payload-specific code.

nearly unchanged at 0.7 s. For this payload, median CPU worker processing stayed below one second; under concurrency, most of the added client-visible delay came from the submit and queue path rather than slower model inference. These measurements characterize only the current CPU deployment on the validation cluster. They are not production service-level objectives, graphics processing unit (GPU) benchmarks, or evidence of HPA scale-out behavior. The Kubernetes manifests nevertheless define HorizontalPodAutoscaler (HPA) behavior for BentoML: when average CPU utilization reaches 70%, the inference deployment can scale from one to three replicas. Because BaseAsyncModelService does not assume CPU execution, GPU-backed serving is a straightforward deployment adaptation rather than a redesign of the service architecture. It requires a CUDA-capable container image, GPU resource requests in the Kubernetes manifests, and corresponding runtime configuration, while leaving the public API contract, queue lifecycle, authentication boundary, and request records unchanged.

## V. A SHARED SERVICE-LAYER PATTERN

Buckets Explorer and the SEM Classifier API address two different operational problems: governed object storage and reusable asynchronous analysis execution. Despite this difference, both implement the same service-layer pattern. Before deployment on ORFEO-facing infrastructure, both services were developed and tested in the Simulated Testing Environment for Nibble Cluster Infrastructure Lab (STENCIL), a virtual data-center environment that reproduced key storage, identity, and Kubernetes paths [36]. STENCIL itself was also deployed on ORFEO. Figure 1 shows the two services side by side as part of a common OFED service-layer catalogue. Figure 6 then abstracts the responsibility split they share: public contract, authentication boundary, control plane, infrastructure data path, and persistent service record. In the storage service, these responsibilities are implemented through Authentik, Django, RGWSquared, Ceph RGW, and Django’s bucket and file tables in PostgreSQL. In the analysis service, they are implemented through Authentik, KrakenD, BentoML, Redis, Kubernetes, and the usage-tracking table in PostgreSQL. For OFED, the practical consequence is that another service layer can reuse the same separation instead of rebuilding policy, execution, and request-record logic from scratch. The same structure could also guide future FAIR-by-design services at other stages of laboratory data pipelines, including data-enrichment steps after acquisition, as a design path for future service layers.

## VI. DISCUSSION AND CONCLUSIONS

**FAIR alignment.** The two service layers support FAIR principles at different points in the OFED lifecycle. Buckets Explorer improves findability by embedding institutional and proposal context in bucket and object names, keeping facility-level provenance visible in ordinary storage listings. Accessibility is provided through S3-compatible retrieval under OIDC/OAuth2-controlled access for governed objects, and through a JWT-authenticated REST contract for analysis requests. Interoperability comes from open, client-facing protocols rather than vendor-specific interfaces. Reusability is

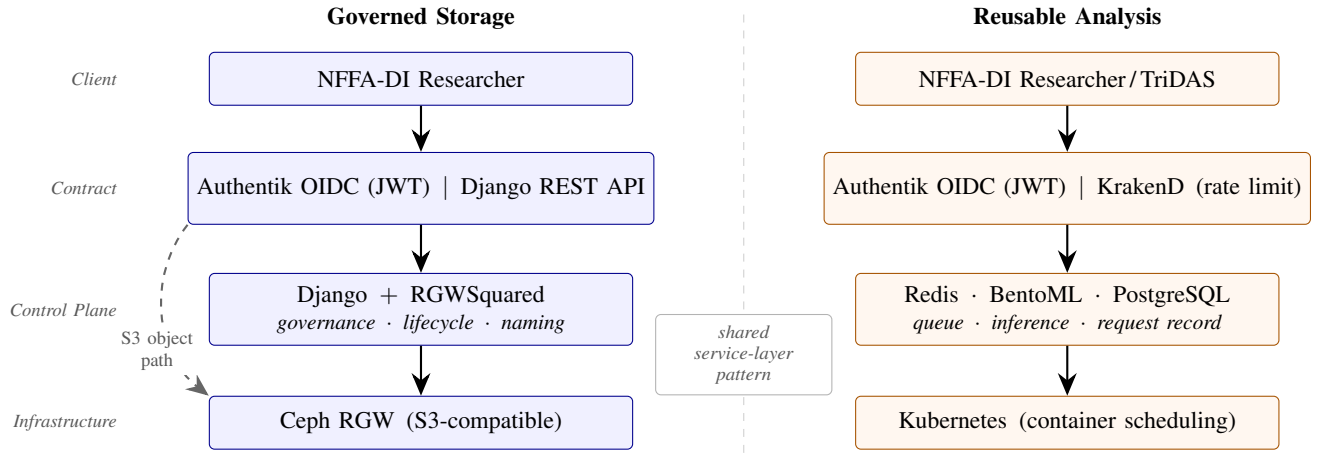


Fig. 6. Shared service-layer pattern used by governed storage (left) and reusable analysis (right). The contract row includes the authentication boundary; the control-plane row applies service rules and retains the persistent service record; the infrastructure row carries object data or computation. The dashed storage arrow denotes S3 object operations that bypass RGWSquared and remain inside the Django-authorized application workflow.

supported primarily by the storage layer, which preserves upload records with the user, bucket, file, and timestamp needed for later governed re-access. The analysis layer records authenticated service usage through the `api_usage` table but does not store outputs or confidence scores, keeping inference separate from result archiving while reducing unnecessary retention of user data.

**Limitations.** The storage and analysis codebases are publicly available [37], [38], but reuse requires site-specific adaptation. Governed storage must be connected to each facility’s tenant model, proposal source, institutional identifiers, identity groups, and governance adapter. For a different inference task, the analysis service requires changes to model loading, input handling, inference logic, and runtime resources, while the gateway, queue lifecycle, authentication boundary, and request-record structure can remain stable.

The contribution is limited to the two deployed service-layer instantiations, the contracts they expose, and the shared pattern demonstrated on NFFA-DI and ORFEO. It does not claim a deployed end-to-end pipeline where storage events trigger analysis jobs and return results to governed storage, nor storage-throughput improvements or a generalized production benchmark across facilities.

**Before and after at ORFEO.** Before this work, ORFEO operators handled storage access and analysis exposure directly around Ceph, Kubernetes, and service-specific code. For storage, this meant translating proposal and tenant rules into Ceph credentials and permissions outside a user-facing governed workspace. For analysis, exposing a model required another service-specific flow for authentication, job state, status, results, and request accounting. Buckets Explorer changes the storage side by giving authorized NFFA-DI users a governed workspace where proposal and tenant rules remain application-visible. The SEM Classifier API changes the analysis side by exposing inference through a deterministic submit-poll-retrieve contract with a per-user request record. Together, the two services show

how facility infrastructure can be wrapped by stable contracts that make policy, access, execution, and records explicit at the service layer.

**Future composition.** These contracts provide a starting point for future composition. Governed objects could later be exposed to cross-site data-management systems such as Rucio, iRODS, or OneData, while the analysis service could become a callable step in TriDAS, Pegasus, JAWS, or another workflow client, including data-enrichment stages in FAIR-by-design laboratory pipelines. A future storage-to-analysis pipeline would still need explicit policies for when storage events may trigger analysis, which user or proposal owns the request, and how result objects return to governed storage. The pattern lets future facility services build from stable contracts, authentication boundaries, governed records, and infrastructure data paths instead of raw infrastructure.

#### ACKNOWLEDGMENT

This work was funded by the FAIR-by-Design project (Programma Nazionale di Ricerca, CUPJ95F21002830001) at AREA Science Park and the joint SISSA–AREA Science Park Master in Data Management and Curation.

The authors thank Marco Cesaratto for developing RGWSquared and the ORFEO operations team for support.

Federica Bazzocchi and Tommaso Rodani contributed equally to supervision.

AI tools supported code development, debugging, background research, and manuscript revision. The authors made all final technical and editorial decisions and remain responsible for the code, sources, analyses, conclusions, and manuscript.

#### REFERENCES

- [1] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg, G. Appleton, M. Axton, A. Baak, N. Blomberg, J.-W. Boiten, L. B. da Silva Santos, P. E. Bourne, J. Bouwman, A. J. Brookes, T. Clark, M. Crosas, I. Dillo, O. Dumon, S. Edmunds, C. T. Evelo, R. Finkers, A. Gonzalez-Beltran, A. J. G. Gray, P. Groth, C. Goble, J. S. Grethe, J. Heringa, P. A. C. Hoen, R. Hooft, T. Kuhn, R. Kok, J. Kok, S. J. Lusher, M. E. Martone, A. Mons,

- A. L. Packer, B. Persson, P. Rocca-Serra, M. Roos, R. van Schaik, S.-A. Sansone, E. Schultes, T. Sengstag, T. Slater, G. Strawn, M. A. Swertz, M. Thompson, J. van der Lei, E. van Mulligen, J. Velterop, A. Waagmeester, P. Wittenburg, K. Wolstencroft, J. Zhao, and B. Mons, "The FAIR guiding principles for scientific data management and stewardship," *Scientific Data*, vol. 3, p. 160018, 2016.
- [2] NFFA-DI, "NFFA-DI consortium partners," NFFA-DI official website, 2026, accessed May 10, 2026. [Online]. Available: <https://nffa-di.it/en/about-us/consortium/>
- [3] F. Bazzocchi, "Data management in NFFA-DI: OFED (Overarching FAIR Ecosystem for Data)," Presentation, Zenodo, 2024. [Online]. Available: <https://zenodo.org/records/13773924>
- [4] AREA Science Park, "ORFEO: High-performance computing and data infrastructure," ORFEO official documentation, 2026, accessed May 2026. [Online]. Available: <https://orfeo-doc.areasciencepark.it/>
- [5] Z. Li, C. Kesselman, M. D'Arcy, M. Pazzani, and B. Y. Xu, "Deriva-ML: A continuous FAIRness approach to reproducible machine learning models," in *2024 IEEE 20th International Conference on e-Science (e-Science)*, 2024, pp. 1–10.
- [6] NOMAD Laboratory, "NOMAD: Open materials-science data platform," NOMAD Laboratory official website, 2026, accessed May 2026. [Online]. Available: <https://nomad-lab.eu/>
- [7] K. Chard, S. Tuecke, and I. T. Foster, "Efficient and secure transfer, synchronization, and sharing of big data," *IEEE Cloud Computing*, vol. 1, no. 3, pp. 46–55, 2014.
- [8] L. Cinquini, D. Crichton, C. Mattmann, J. Harney, G. Shipman, F. Wang, R. Ananthakrishnan, N. Miller, S. Denvil, M. Morgan, Z. Pobre, G. Bell, C. Doutriaux, R. Drach, D. Williams, P. Kershaw, S. Pascoe, E. Gonzalez, S. Fiore, and R. Schweitzer, "The Earth System Grid Federation: An open infrastructure for access to distributed geospatial data," *Future Generation Computer Systems*, vol. 36, pp. 400–417, 2014.
- [9] M. Barisits, T. Beermann, F. Berghaus, B. Bockelman, J. Bogado, D. Cameron, D. Christidis, D. Ciangottini, G. Dimitrov, M. Elsing, V. Garonne, A. di Girolamo, L. Goossens, W. Guan, J. Guenther, T. Javurek, D. Kuhn, M. Lassnig, F. Lopez, N. Magini, A. Molfetas, A. Nairz, F. Ould-Saada, S. Prenner, C. Serfon, G. A. Stewart, E. Vaandering, P. Vasileva, R. Vigne, and T. Wegner, "Rucio: Scientific data management," *Computing and Software for Big Science*, vol. 3, no. 1, p. 11, 2019.
- [10] A. Rajasekar, R. W. Moore, C.-Y. Hou, C. A. Lee, R. Marciano, A. de Torcy, M. Wan, W. Schroeder, S.-Y. Chen, L. Gilbert, P. Tooby, and B. Zhu, "iRODS primer: Integrated rule-oriented data system," *Synthesis Lectures on Information Concepts, Retrieval, and Services*, vol. 2, no. 1, pp. 1–143, 2010.
- [11] L. Dutka, R. Słota, M. Wrzeszcz, D. Król, and J. Kitowski, "Uniform and efficient access to data in organizationally distributed environments," in *eScience on Distributed Computing Infrastructure*, ser. Lecture Notes in Computer Science, M. Bubak, J. Kitowski, and K. Wiatr, Eds. Cham: Springer, 2014, vol. 8500, pp. 178–194.
- [12] E. Deelman, K. Vahi, G. Juve, M. Rynge, S. Callaghan, P. J. Maechling, R. Mayani, W. Chen, R. Ferreira da Silva, M. Livny, and K. Wenger, "Pegasus, a workflow management system for science automation," *Future Generation Computer Systems*, vol. 46, pp. 17–35, 2015.
- [13] D. Cassol, A. Clum, J. Froula, E. Kirton, R. Kothadia, M. Melara, E. Player Jackson, S. Sarafan, S.-J. Sul, S. Trong, N. Tyler, T. Bruna, L. Baumgart, and K. Fagnan, "Supporting FAIR scientific workflows with the JGI analysis workflow service (JAWS)," in *2025 IEEE 21st International Conference on e-Science (e-Science)*, 2025, pp. 141–149.
- [14] S. Park, R. Q. Thomas, C. C. Carey, A. D. Delany, Y.-J. Ku, M. E. Lofton, and R. J. Figueiredo, "FaaSr: Cross-platform function-as-a-service serverless scientific workflows in R," in *2024 IEEE 20th International Conference on e-Science (e-Science)*, 2024, pp. 1–10.
- [15] M. E. Pierce, S. Marru, L. Gunathilake, T. A. Kanewala, R. Singh, S. Wijeratne, C. Wimalasena, C. Herath, E. Chinthaka, C. Mattmann, A. Slominski, and P. Tangchaisin, "Apache Airavata: Design and directions of a science gateway framework," *Concurrency and Computation: Practice and Experience*, vol. 27, no. 16, pp. 4282–4291, 2015.
- [16] M. S. Islam, T. Azaz, A. Raza, A. S. M. S. Hossain, F. Baig, S. Wang, K. Lannon, T. Malik, and D. Thain, "Backpacks for notebooks: Enabling containerized notebook workflows in distributed environments," in *2025 IEEE 21st International Conference on e-Science (e-Science)*, 2025, pp. 169–177.
- [17] A. Withers, B. Bockelman, D. Weitzel, D. Brown, J. Gaynor, T. Tanenbaum, Z. Miller, and J. Basney, "SciTokens: Capability-based secure access to remote scientific data," in *Proceedings of the Practice and Experience on Advanced Research Computing (PEARC '18)*, 2018.
- [18] NFFA-DI, "Data infrastructure," NFFA-DI official website, 2025, accessed April 20, 2026. [Online]. Available: <https://nffa-di.it/en/data-infrastructure/>
- [19] N. Sakimura, J. Bradley, M. B. Jones, B. de Medeiros, and C. Mortimore, "OpenID Connect Core 1.0 incorporating errata set 1," OpenID Foundation specification, 2014. [Online]. Available: [https://openid.net/specs/openid-connect-core-1\\_0.html](https://openid.net/specs/openid-connect-core-1_0.html)
- [20] D. Hardt, "The OAuth 2.0 authorization framework," Internet Engineering Task Force, RFC 6749, Oct. 2012. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6749>
- [21] M. B. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Internet Engineering Task Force, RFC 7519, May 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>
- [22] authentik, "Welcome to authentik," authentik official documentation, 2026, accessed April 20, 2026. [Online]. Available: <https://docs.goauthentik.io/>
- [23] S. A. Weil, S. A. Brandt, E. L. Miller, D. D. E. Long, and C. Maltzahn, "Ceph: A scalable, high-performance distributed file system," in *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI '06)*. USENIX Association, 2006, pp. 307–320.
- [24] Ceph, "Ceph object gateway," Ceph official documentation, 2026, accessed April 20, 2026. [Online]. Available: <https://docs.ceph.com/en/squid/radosgw/>
- [25] NFFA-DI, "Research data policy," NFFA-DI official website, 2025, last updated March 25, 2025; accessed April 20, 2026. [Online]. Available: <https://nffa-di.it/en/research-data-policy>
- [26] Amazon Web Services, "What is Amazon S3?" Amazon S3 official documentation, 2026, accessed April 20, 2026. [Online]. Available: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html>
- [27] NFFA-Europe, "TriDAS: Trieste advanced data services," NFFA-Europe official website, 2026, accessed May 2026. [Online]. Available: <https://nffa.eu/apply/virtual-access-services/tridas/>
- [28] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations (ICLR)*, 2021. [Online]. Available: <https://openreview.net/forum?id=YicbFdNTTy>
- [29] R. Aversa, M. H. Modarres, S. Cozzini, R. Ciancio, and A. Chiusole, "The first annotated set of scanning electron microscopy images for nanoscience," *Scientific Data*, vol. 5, p. 180172, 2018.
- [30] T. Rodani, "vit-sem-classification: Vision Transformer for SEM image classification," Hugging Face model repository, 2025, apache-2.0 license. Accessed May 2026. [Online]. Available: <https://huggingface.co/t0m-R/vit-sem-classification>
- [31] BentoML, "BentoML documentation," BentoML official documentation, 2026, accessed April 20, 2026. [Online]. Available: <https://docs.bentoml.com/>
- [32] Redis, "Open source," Redis official documentation, 2026, accessed April 20, 2026. [Online]. Available: <https://redis.io/docs/latest/get-started/>
- [33] T. Rodani, "vit-sem-scale-classifier: Vision Transformer for SEM scale classification," Hugging Face model repository, 2025, apache-2.0 license. Accessed May 2026. [Online]. Available: <https://huggingface.co/t0m-R/vit-sem-scale-classifier>
- [34] T. Rodani, "vit-stm-artifact-fft: Vision Transformer for STM multi-tip artifact detection," Hugging Face model repository, 2025, apache-2.0 license. Accessed May 2026. [Online]. Available: <https://huggingface.co/t0m-R/vit-stm-artifact-fft>
- [35] T. Rodani, A. Ansuini, and A. Cazzaniga, "Enhancing multi-tip artifact detection in STM images using fourier transform and vision transformers," in *ICML 2024 Workshops: ML4LMS*, 2024. [Online]. Available: <https://mlanthology.org/icmlw/2024/rodani2024icmlw-enhancing/>
- [36] I. Pasianotto, "STENCIL: Simulated testing environment for nibble cluster infrastructure lab," Project webpage, 2026, accessed May 17, 2026. [Online]. Available: <https://isacpasianotto.com/projects/stencil/>
- [37] L. F. Palacios Flores and F. Bazzocchi, "Bucket Explorer," GitHub repository, 2026, version 1.0.0, released May 26, 2026; EUPL-1.2 license. Accessed May 2026. [Online]. Available: <https://github.com/RitAreaSciencePark/s3bucket-manager-app>
- [38] L. F. Palacios Flores and T. Rodani, "Reusable Async Analysis Service Pipeline," GitHub repository, 2026, version 1.0.0, released May 26, 2026; EUPL-1.2 license. Accessed May 2026. [Online]. Available: <https://github.com/RitAreaSciencePark/sem-image-classifier-api>